Avinash Group ERP — Technical Reference

Developer reference for `avinashgroup_app` : custom Script Reports (brief) and the customizations / portal pages (detailed). Reflects current code.

Quick Reference — Reports

#	Report	Source	Purpose
1	Party Ledger	GL Entry	Tally-style party statement: opening → txns → closing.
2	Party Ledger Summary	GL Entry	One line/party: opening / debit / credit / closing.
3	Sales Register	Sales Invoice	Per-invoice VAT register (tax-free / export / taxable / VAT).
4	Purchase Register	Purchase Invoice	Per-invoice VAT register (tax-free / taxable / import / capitalized).
5	Receipt Register	Payment Entry	Customer receipts in 3 views.
6	SA — Customer Summary	Sales Invoice	Per-customer qty/value, optional returns/net.
7	SA — Customer Details	Sales Invoice	Per customer → product (UOM).
8	SA — Product Details	Sales Invoice	Per product → invoice → customer.
9	Advance Tax TDS	Purchase Invoice	Supplier TDS withholding by category.
10	Loan Summary	GL Entry	Company-wise loan closing balances.
11	One Lakh Above	SI + PI	Parties with totals ≥ NPR 100,000.

Jump: Customizations & Portal · Shared Architecture (`*` = required filter; PDF patterns A/B/C defined in Shared Architecture.)

Reports

1. Party Ledger

- **What:** Ledger for one party, or grouped multi-party when 0 / 2+ selected. Powers `/customer_statement`.
- **Source:** `tabGL Entry` (`is_cancelled=0`, company, party_type); `LEFT JOIN` SI/PI for description. Running balance in Python; rows merged per voucher.
- **Filters:** `company*`, `party_type`, `party` (1→flat, 0/2+→grouped), `account`, `from*` / `to*`, `voucher_no` (LIKE), `show_remarks`, `show_balance`, `detailed_mapping`, `show_contract_form`.
- **PDF:** A — `party_ledger_pdf.html`, default Landscape, Pick-Columns.

> ⚠️ One row/voucher × party → large for wide ranges; enrichment batched (`IN` chunks of 500).

2. Party Ledger Summary

- **What:** One aggregate line/party. Modes: **Super Summary** (flat + grand total) / **Group Wise** (per-group subtotals).
- **Source:** `tabGL Entry GROUP BY party`; `opening = SUM(debit-credit) WHERE posting_date < from_date OR is_opening='Yes'`.
- **Filters:** `report_type*`, `company*`, `party_type*`, `party_group`, `party`, `from*` / `to*`, `show_zero_balance`, `closing_drcr` (DB/CR).
- **PDF:** A — default Portrait. **Excel export overridden**: Opening/Closing as magnitude + separate Dr/Cr column.

3. Sales Register

- **What:** One row / submitted Sales Invoice; VAT split tax-free / export / taxable / VAT. Returns view (`abs()`).
- **Source:** `SI ⋈ SII ⋈ Customer`, `GROUP BY si.name`. Buckets = conditional `SUM` over `sii.custom_vat_apply_on` + `c.territory`.
- **Filters:** `company`, `from*` / `to*`, `customer`, `is_return`.
- **PDF:** A — Landscape, Pick-Columns.

> ⚠️ `company` / `customer` string-interpolated into SQL; dates bound.

4. Purchase Register

- **What:** One row / Purchase Invoice; buckets tax-free / taxable / import / capitalized + VAT + qty.
- **Source:** `PI ⋈ PII ⋈ Supplier ⋈ Item`, `GROUP BY pi.name`. Buckets via `custom_vat_apply_on` + `is_fixed_asset` + `custom_territory`.
- **Filters:** `company`, `from*` / `to*`, `supplier`, `purchase_type`, `is_return`.
- **PDF:** A — Landscape; shares the Pick-Columns patch with Sales Register.

5. Receipt Register

- **What:** Customer receipts (Payment Entry, `payment_type='Receive'`). 3 views: Date Wise / Customer Wise / Summary.
- **Source:** `tabPayment Entry` only; `docstatus=1`, `party_type='Customer'`.
- **Filters:** `view*`, `company*`, `from*` / `to*`, `customer`, `bank`.
- **PDF:** A — orientation auto per view.

> ⚠️ "GL Code" column is an unfinalized placeholder; cheque no `"1"` suppressed.

6. Sales Analysis — Customer Summary

- **What:** Per-customer qty + gross value (incl. excise); optional returns/net.
- **Source:** `SI ⋈ SII ⋈ Customer`, `GROUP BY customer`. `value = SUM(amount + custom_excise_value)`; returns negated to positive.
- **Filters:** `company`, `from*` / `to*`, `customer`, `customer_group`, `item_code`, `include_return`.

- **PDF:** A — Landscape.

> ⚠️ `item_code` narrows lines but grouping stays per customer.

7. Sales Analysis — Customer Details

- **What:** Per customer → product (UOM): qty / value / VAT / total-incl-VAT + rollups.
- **Source:** `SII ⋈ SI ⋈ Item ⋈ Customer`, `GROUP BY customer, item_code, uom, is_return`.
- **Filters:** `company`, `from*` / `to*`, `customer`, `item_code`, `include_return`.
- **PDF:** A — Portrait.

> ⚠️ Granularity customer × item × UOM → row count can grow large.

8. Sales Analysis — Product Details

- **What:** Per product → invoice → customer, with returns/net.
- **Source:** `GROUP BY item_code, uom, customer, si.name, is_return`.
- **Filters:** `company`, `from*` / `to*`, `customer`, `item_code`, `include_return`.
- **PDF:** A — Landscape.

> ⚠️ No agent data source → always "No Agent". `include_return` unchecked never reaches server (Frappe drops falsy filters → treated OFF).

9. Advance Tax TDS Details

- **What:** Supplier TDS withholding statement by category (Devanagari headers); per-section + grand totals.
- **Source:** `PI ⋈ PII ⋈ Supplier`. `turnover = SUM(amount WHERE apply_tds=1)`, `tds = SUM(custom_tds_amount)`, `GROUP BY supplier, category, tax_id`. Category string parsed (regex) → rate / khata / title.
- **Filters:** `company*`, `from*` / `to*`.
- **PDF:** B — shared helper + Prepared/Checked/Verified signature footer (print only).

10. Loan Summary

- **What:** Company-wise loan closing balances (short/long-term borrowings); ratio row; `Show Details` expands sub-accounts.
- **Source:** `tabAccount` (nested-set, CoA groups) → `tabGL Entry`: `SUM(credit-debit) WHERE posting_date <= to_date`.
- **Filters:** `company`, `to_date*`, `show_details`.
- **PDF:** B — shared helper.

> ⚠️ `from_date` is collected but **never used** — balances are cumulative to To Date, not a range.

11. One Lakh Above Transactions

- **What:** Parties (customers + suppliers) with taxable/exempt totals ≥ NPR 100,000.
- **Source:** SI and PI queries concatenated; `taxable = SUM(ABS(amount)) WHERE custom_vat_amount!=0`; `HAVING ≥ 100000` (+ re-checked in Python).
- **Filters:** `company*`, `from*` / `to*`.
- **PDF:** C — stock Frappe print.

> ⚠️ `trade_name_type` hardcoded `'E'`; no totals row.

Customizations & Portal

Branch-wise Warehouse Auto-fetch

Purpose: Auto-set the line Warehouse on buying/selling docs to the correct branch warehouse, so one item carries different stock warehouses per branch (and buying vs selling).

How it works: - Mapping lives on **Item** child table `custom_branch_wise_warehouse` rows: `custom_branch`, `custom_buying_warehouse`, `custom_selling_warehouse`; Item-level `custom_buying_warehouse` / `custom_selling_warehouse` are fallbacks. - Resolution per item-add: branch row (matching the document's `custom_branch`) → Item-level fallback → leave whatever ERPNext set (never blanks an existing value). - **Selling** (SI + Quotation / SO / DN): on `item_code`, temporarily wraps `frappe.call` so ERPNext's `get_item_details` response `warehouse` is replaced with the branch-resolved selling warehouse before it's written (avoids a race; auto-restores after the call / 5s). A `before_save` sweep re-forces every row; the SI `set_warehouse` header handler re-applies on header change. - **Buying / Material Request:** overrides `frm.events.get_item_data` to force the branch-resolved buying warehouse.

Files: `doctype/branch_wise_warehouse/` (child table) · `public/js/sales_invoice.js` (`_fetch_selling_wh`, `force_all_si_warehouses`) · `sales_warehouse_common.js` · `material_request.js` · `hooks.py` (`app_include_js`, `doctype_js`).

Place Order — Customer Portal (`/place_order`)

Purpose: Logged-in customer self-service to place a Sales Order for **LP Gas**.

Flow: - **Auth:** Guests → `/login`; `PermissionError` unless `Customer` role. Customer resolved from the `Portal User` child table (linked customers render as `<select>` filtered by company; otherwise free-text `search_customers`, company-scoped). - **Context:** default company (user default → Global Defaults), currency, today, delivery date = today + 7. Fixed **LP Gas** item (read-only); UOM restricted to `5 KG` / `7.1 KG` / `14.2 KG`. - **Pricing:** `get_customer_defaults` (price list / currency / `custom_company`); `get_item_price` reads `Item Price.price_list_rate`. Client computes amount = qty × rate, VAT per row (13% / 0% / manual), totals + amount-in-words. - **Submit:** `create_sales_order` re-checks role/customer, validates rows (item, qty>0, delivery date), builds & submits a **Sales Order** (`order_type='Sales'`, conversion rates 1.0, `ignore_permissions`, `owner` = session user) → redirect `/orders`.

Files: `templates/pages/place_order.py` (`get_context`; whitelisted `create_sales_order`, `get_customer_defaults`, `get_item_price`, `search_*`) · `place_order.html` (inline `PlaceOrder` JS).

Customer Statement — Customer Portal (`/customer_statement`)

Purpose: Logged-in customer views their own account statement (Party-Ledger ledger) and downloads a Portrait PDF.

Flow: - **Auth/scope:** `Customer` role required. `_get_portal_customers` (Portal User table) + `_get_allowed_companies` (their `custom_company`) drive the company dropdown and customer checkboxes. - **Server re-validation:** every AJAX call passes through `_resolve_request`, which rejects companies/customers outside the user's set and **fills the party list with the user's customers when none are chosen** — an empty party list would mean "all parties" in Party Ledger and leak other customers' data. - **Reuse:** imports Party Ledger `execute` / `download_pdf`. `get_statement` builds filters (`party_type='Customer'`, validated `party`, company, dates, `detailed_mapping=0`, `show_remarks=0`) → `execute`. One customer → flat layout (+ PAN/VAT); >1 → grouped (`multi_customer = len != 1`). - **Dates:** paired AD + Nepali BS pickers (AD is source of truth), debounced reload (~350 ms). - **PDF:** `download_pdf` re-runs `_resolve_request`, calls Party Ledger PDF with Portrait, `capacity_override=76`.

Files: `templates/pages/customer_statement.py` (`get_context`, `_get_portal_customers`, `_resolve_request`, `get_statement`, `download_pdf`) · `customer_statement.html` (inline JS) · reuses `report/party_ledger/party_ledger.py`.

Request for Quotation — Supplier Portal override (`/rfq/<name>`)

Purpose: Supplier enters rate / VAT / discount / attachments and submits a Supplier Quotation; the override adds data ERPNext's default doesn't handle.

How it works: - `get_context` delegates to ERPNext (renders local `rfq.html`). Attachments upload immediately as private Files via whitelisted `upload_portal_item_attachment`. - `hooks.py` maps ERPNext `create_supplier_quotation` → the local override, so the existing client call hits the custom code. **vs ERPNext default it adds:** - document-level discounts (`apply_discount_on`, `additional_discount_percentage`, `discount_amount`); - per-line VAT custom fields (`custom_vat_apply_on` / `_rate` / `_amount`, guarded by `item_meta.has_field`); - `calculate_taxes_and_totals` after `set_missing_values`; - re-links uploaded Files (`attached_to_doctype` / `_name`) to the new Supplier Quotation.

Files: `templates/pages/rfq.py` (`create_supplier_quotation`, `_add_items`, `upload_portal_item_attachment`, `_attach_uploaded_files_to_supplier_quotation`) · `rfq.html` · `hooks.py` (`override_whitelisted_methods`).

Company-wise Filtering

Purpose: Restrict Link-field dropdowns (incl. child-table & Dynamic Link) to the document's company; block cross-company saves.

How it works: - **Config:** `Company Filter Config` (`doctype_name`, `company_field`) + child `Company Filter Field` (`fieldname`, `is_child_table` / `child_fieldname`, `is_dynamic_link` / `dynamic_link_field`). Hardcoded `FILTER_CONFIG` is the pre-migrate fallback. - **Serve:** `get_filter_config` (Redis-cached) reshapes config + pre-resolves a `linked_doctype → company field` map. - **Client:** `company_filter.js` registers per-doctype events; `frm.set_query()` filters each field; Dynamic Link delegates to `search_link_by_company`. On company change, mismatched values cleared / child rows removed. - **Server enforce:** `validate_company_matching` runs a 3-phase `CompanyValidator` (collect → batch query → compare) → throws "Company Mismatch" (warn-only on import). - **Cache:** cleared on Config/Field save/delete.

Files: `custom_code/globalfilter/globalfilter.py` · `public/js/company_filter.js`, `global_filter.js` · doctypes `Company Filter Config` / `Company Filter Field` · `hooks.py`.

Credit Control (Sales Invoice)

Purpose: Block new invoices for customers over their credit limits (overdue days, outstanding amount, unpaid-bill count).

How it works: Reads Customer limits. On customer change → days + bill-count check (borderline date = `today − days`, outstanding from `Customer Ledger Summary`). On `before_save` → adds amount check (existing outstanding + this invoice's grand total vs limit; current invoice excluded from the unpaid query). Client shows a warning dialog and rejects the save promise when `is_blocked`.

Files: `custom_code/SalesInvoice/salesinvoice_customer.py` (whitelisted `check_customer_credit_limit_on_load` / `_on_save`) · `public/js/si.js`.

> ⚠️ Enforced via the client `before_save` path — the SI `validate` doc_event points at the taxes handler, not the credit `validate`. An unused `credit_control.py` variant exists.

Automatic Due Date (Sales Invoice)

Purpose: `due_date = posting_date + Customer.custom_days_limit`.

How it works: Client-side only. `set_due_date_from_customer` runs on new-invoice refresh and on `customer` / `posting_date` change (wrapped in `setTimeout(…,0)` so it wins over ERPNext's core handler). Missing limit → 0 days (due = posting).

Files: `public/js/sales_invoice.js`.

Selling VAT & Tax Handling

Purpose: Consistent VAT + excise across SI / Quotation / SO / DN with default 13%.

How it works: - Per line `custom_vat_apply_on` ∈ {`VAT 13%`, `VAT 0%`, `Amount`} (default 13%). `custom_total = base_net_amount + custom_excise_value`. - VAT: 13% computed, 0% zeroed, Amount manual (rate forced 0). - `update_taxes_table` finds Excise (acct prefix `348204`) + VAT (prefix `VAT`), writes/updates `Actual` rows, then `calculate_taxes_and_totals`. - Returns: qty forced negative (`before_validate`), `custom_vat_amount` forced `-abs()` (last in `before_save`); client mirrors on edit.

Files: `custom_code/common/selling_taxes_handler.py`, `custom_code/SalesInvoice/salesinvoice_taxes.py` · `public/js/selling_taxes_common.js`, `sales_invoice.js` · `hooks.py` doc_events.

Purchase VAT & TDS Handling

Purpose: VAT + Excise + TDS on PI / PO / PR / Supplier Quotation; warehouse defaulting on MR / RFQ.

How it works: - VAT identical to selling side. **TDS** via `custom_tds_apply_on` (Percentage / Amount); applies only when `custom_tax_withholding_category_custom` set AND row `apply_tds`. Percentage pulls the rate from the Tax Withholding Category. - `update_taxes_table` adds Excise/VAT (`Add`) and TDS (`Deduct`) `Actual` rows (TDS acct from the custom category's company row); stale TDS rows removed; then `calculate_taxes_and_totals`. - **Cross-document mapping** (`purchase_taxes_mapper.py`): copies item/doc custom tax fields SQ→PO→PR/PI. - Uses a **separate** `custom_tax_withholding_category_custom` field so ERPNext's native TDS engine stays out.

Files: `custom_code/common/purchase_taxes_handler.py`, `purchase_taxes_mapper.py` · `public/js/purchase_taxes_common.js`, `pi.js` · `hooks.py` doc_events (incl. `validate_material_request`, `validate_request_for_quotation`).

Automatic Document Numbering (Voucher Number Settings)

Purpose: Human-facing voucher number `custom_document_no` + composite `custom_name` (e.g. `SGU-RC-000006-82/83`), auto or manual per transaction type.

How it works: - `AUTO_NUMBER_CONFIG` (discriminator field + qualifying type values) decides auto vs manual. - Next number = `max(matching custom_document_no) + 1`, matched by a `custom_name` LIKE pattern (`company_abbr` + `p_type_code` + fiscal year). User-typed value preserved. - Zero-pad width from **Voucher Number Settings Item** (`doctype_name → voucher_no_digits`, default 6). `custom_name = {abbr}-{p_type_code}-{padded_no}{word}-{fiscal_year}`; uniqueness validated. - Client (`auto_update_document_no.js`)

calls whitelisted `get_next_custom_document_no` on new-form load / type / scope change; server re-runs authoritatively (via `audit_file_manager` dispatchers).

Files: `custom_code/Override/naming_series.py` · `public/js/auto_update_document_no.js` · `utils/audit_file_manager.py` · doctype `Voucher Number Settings` (+ child `Voucher Number Settings Item`).

Item Price Company Auto-assignment

Purpose: Stamp `company` on Item Price records auto-created by ERPNext, so price lists stay company-scoped (prevents auto-insert failures when the field is mandatory).

How it works: Monkey-patches `erpnext.stock.get_item_details.insert_item_price`, preserving its standard logic but writing `company` (and/or `custom_company`, whichever the meta has) from the transaction `args`. One-time guard; wired via `before_request`.

Files: `custom_code/Override/auto_insert_item_price.py` · `hooks.py` (`before_request`).

Workflow Reject Reason

Purpose: Force a written reason on the workflow **Reject** action (audit trail).

How it works: `before_workflow_action` opens a mandatory "Rejection Reason" dialog only for Reject; on submit calls whitelisted `set_reject_reason` → writes `custom_reason` (if the field exists, e.g. Purchase Order) else a document comment. `approval_workflow_auto.js` auto-binds it to any Dynamic-Approval doctype (skips ones with their own handler, like Material Request).

Files: `custom_code/workflow.py`, `workflow_material_request.py` · `public/js/approval_workflow_common.js`, `approval_workflow_auto.js`, `material_request.js` · `custom/purchase_order.json` (`custom_reason`).

Shared Architecture

Execution

- All **Script Reports** (`is_standard: "Yes"`); `execute(filters)` builds rows in Python; most have `add_total_row=0` (manual bold total rows).
- `prepared_report=0` → run **synchronously** in the web worker.

> ⚠️ Frappe auto-enables `prepared_report` if `execute()` exceeds ~15s. Loan Summary queries `tabAccount` first so `tabGL Entry` is hit only with an indexed `account IN (...)` .

Company scoping

Filter options come from whitelisted helpers (`get_company_customers` , `get_company_suppliers` , `get_company_items` , `get_company_customer_groups` , `get_company_party_groups` , `get_company_bank_accounts`) scoped via `custom_company` (usually allowing blank).

BS / Miti dates

Nepali "Miti" derived per doctype from `custom_invoice_miti` / `custom_nepali_miti` / `custom_posting_miti` , typically `SUBSTRING_INDEX(..., ' ', 1)` .

Number formatting

`_fmt_inr` — Indian lakh/crore grouping (`en_IN` locale, fallback `{:,.2f}`), blank for zero/None. `_fmt_qty` — 3 dp. Currency labelled NPR.

PDF patterns

Pattern	Reports	Mechanism
A Custom template	Party Ledger, Party Ledger Summary, Sales/ Purchase Register, Receipt Register, SA ×3	Whitelisted `download_pdf` re-runs `execute` , paginates in Python, renders `*_pdf.html` , `frappe.utils.pdf.get_pdf` .
B Shared helper	Advance Tax TDS, Loan Summary	`public/js/report_print_orientation.js` (`app_include_js`): orientation dialog, portrait CSS, `render_pdf` black-grid. No local template.
C Stock Frappe	One Lakh Above	Built-in query-report Print / PDF / Export.

Pattern A traits: in-body `Page X of Y` (works on plain/unpatched wkhtmltopdf, no `--footer-html`); manual pagination by per-orientation row capacity (`brk` = `page-break-after: always`); Print overridden → calls `download_pdf` with `view=1` (inline, identical PDF); column selection via the print dialog's "Pick Columns".